PROGRESS IV
KONTROL MOTOR NIRKABEL DENGAN OTOMASI TIMER COUNTDOWN DAN PEMBARUAN FIRMWARE OVER-THE-AIR (OTA)

4.1 Tujuan Modul

Tahap Progress 4 bertujuan untuk mengintegrasikan fitur *Over-The-Air* (OTA) ke dalam arsitektur kontrol nirkabel Blynk dan algoritma *dynamic timer* yang telah dibangun sebelumnya. Pengujian difokuskan pada kemampuan mikrokontroler ESP32 dalam menerima pembaruan program (*firmware*) secara nirkabel melalui jaringan Wi-Fi, tanpa membutuhkan koneksi kabel serial fisik (USB), sembari tetap mempertahankan fungsionalitas otomasi hitung mundur dan kontrol PWM secara stabil.

4.2 Skema Rangkaian

Skema pengkabelan pada tahap ini bersifat final dan tidak mengalami modifikasi perangkat keras (*hardware*). ESP32 tetap terhubung dengan driver L298N untuk mengendalikan aktuator motor, dengan konfigurasi pin yang identik:

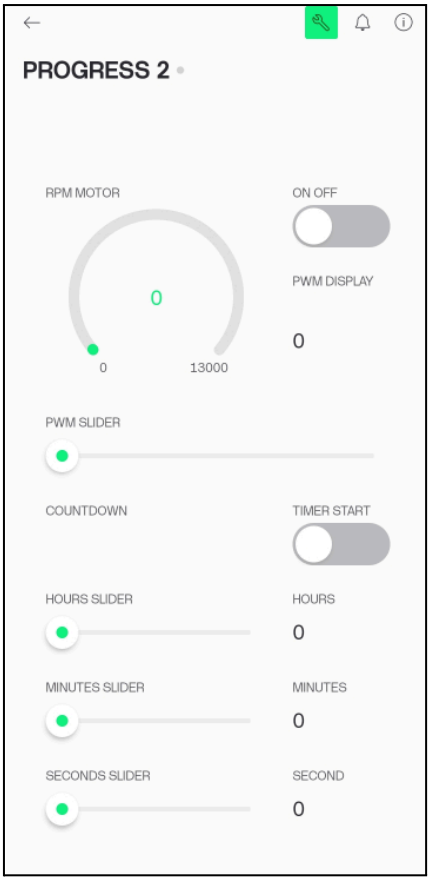
- Pin 27 (motor1Pin1): Dihubungkan ke pin IN1 driver motor.
- Pin 26 (motor1Pin2): Dihubungkan ke pin IN2 driver motor.
- Pin 12 (enable1Pin): Dihubungkan ke pin ENA driver motor (sumber sinyal PWM).

4.3 Konsep & Logika Sistem OTA

Sistem OTA (*Over-The-Air*) mengubah paradigma pemeliharaan perangkat IoT. Biasanya, pembaruan (*flashing*) kode mengharuskan perangkat dihubungkan ke komputer via kabel. Dengan menyuntikkan pustaka ArduinoOTA ke dalam program dasar, ESP32 akan mengekspos dirinya (*broadcasting*) di dalam Local Area Network (LAN). Selama ESP32 dan komputer (Arduino IDE) berada di dalam SSID Wi-Fi yang sama, IDE dapat mendeteksi "Network Port" mikrokontroler. Modul ini membuktikan bahwa siklus instruksi *timer* hitung mundur (millis()) dan protokol komunikasi Blynk tidak mengalami disrupsi saat port OTA aktif dalam status *standby* untuk menerima transmisi binari (*firmware*).

4.4 Konfigurasi Widget

Karena modul OTA ini merupakan peningkatan (*upgrade*) internal perangkat lunak ESP32, antarmuka pengguna di sisi klien (*smartphone*) tidak memerlukan modifikasi. Proyek ini menggunakan ulang arsitektur antarmuka dan *Virtual Pin* (V0 hingga V8) secara utuh dari Progress 3.



Gambar 1. Konfigurasi dan integrasi tata
letak *widget dynamic timer* pada kanvas aplikasi Blynk.

4.5 Source Code dan Penjelasan Logika Program

```
// PROGRESS 4 : FULL BLYNK + DYNAMIC TIMER + OTA
// Kontrol Jarak Jauh & Update Nirkabel

// --- KREDENSIAL BLYNK & WIFI ---
#define BLYNK_TEMPLATE_ID "ISI_DENGAN_TEMPLATE_ID_ANDA"
#define BLYNK_TEMPLATE_NAME "ISI_DENGAN_NAMA_TEMPLATE_ANDA"
#define BLYNK_AUTH_TOKEN "ISI_DENGAN_AUTH_TOKEN_DEVICE_BARU"
#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <ESPmDNS.h>
#include <WiFiUdp.h>
#include <ArduinoOTA.h>

// Masukkan SSID dan Password Wi-Fi/Tethering Anda
```

```
char ssid[] = "NAMA_WIFI_ANDA";
char pass[] = "PASSWORD_WIFI_ANDA";

// --- PIN MOTOR ---
int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// --- PENGATURAN PWM ---
const int freq = 30000;
const int resolution = 8;
int dutyCycle = 0;
bool isMotorRunning = false;

// --- VARIABEL DYNAMIC TIMER ---
int inputJam = 0;
int inputMenit = 0;
int inputDetik = 0;

unsigned long waktuMulaiTimer = 0;
unsigned long durasiTimer = 0;
bool isTimerAktif = false;
long sisaWaktuTerakhir = -1;

unsigned long waktuTeksTerakhir = 0;
const long jedaTeks = 5000;

// Fungsi Kontrol Motor & Update Ke BLYNK

void applyMotor() {
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);

    Blynk.virtualWrite(V1, dutyCycle); // V1: PWM DISPLAY

    int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
    Blynk.virtualWrite(V0, simulasiRPM); // V0: RPM MOTOR

  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);

    Blynk.virtualWrite(V1, 0);
    Blynk.virtualWrite(V0, 0);
  }
}
```

```
// --- V3: KONTROL MANUAL ON/OFF ---
BLYNK_WRITE(V3) {
  if (param.asInt() == 1) {
    isMotorRunning = true;
    isTimerAktif = false; // Batalkan timer jika dinyalakan manual
    if (dutyCycle == 0) dutyCycle = 150;
    Serial.println(">>> [BLYNK] Motor ON");
  } else {
    isMotorRunning = false;
    isTimerAktif = false;
    Serial.println(">>> [BLYNK] Motor OFF");
  }
  applyMotor();
}

// --- V2: SLIDER NAIK/TURUN KECEPATAN ---
BLYNK_WRITE(V2) {
  dutyCycle = param.asInt();
  if (isMotorRunning) applyMotor();
}

// --- MEMBACA SLIDER WAKTU (V4, V5, V6) ---
BLYNK_WRITE(V4) { inputJam = param.asInt(); }
BLYNK_WRITE(V5) { inputMenit = param.asInt(); }
BLYNK_WRITE(V6) { inputDetik = param.asInt(); }

// --- V7: TOMBOL TIMER START ---
BLYNK_WRITE(V7) {
  if (param.asInt() == 1) {
    // Kalkulasi total waktu ke dalam milidetik
    durasiTimer = (inputJam * 3600UL + inputMenit * 60UL + inputDetik) *
1000UL;

    if (durasiTimer > 0) {
      isTimerAktif = true;
      isMotorRunning = true;
      waktuMulaiTimer = millis();
      sisaWaktuTerakhir = -1;

      if (dutyCycle == 0) dutyCycle = 150;
      applyMotor();

      Serial.println("\n>>> [TIMER] Memulai hitung mundur!");
    } else {
      // Jika tombol ditekan tapi slider waktu masih 0
      Blynk.virtualWrite(V7, 0);
      Blynk.virtualWrite(V8, "Atur Waktu!");
    }
```

```
  } else {
    // Jika tombol dimatikan di tengah jalan
    isTimerAktif = false;
    isMotorRunning = false;
    applyMotor();
    Blynk.virtualWrite(V8, "BATAL");
    Serial.println(">>> [TIMER] Dibatalkan pengguna.");
  }
}

//Setup Awal

void setup() {
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  isMotorRunning = false;
  applyMotor();

  // 1. Konek ke WiFi
  Serial.println("\n Booting WiFi...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, pass);
  while (WiFi.waitForConnectResult() != WL_CONNECTED) {
    Serial.println("Koneksi WiFi Gagal! Restarting...");
    delay(3000); ESP.restart();
  }
  Serial.println("WiFi Terhubung!");
  Serial.print("IP Address ESP32: ");
  Serial.println(WiFi.localIP());

  // 2. Hubungkan ke Server Blynk
  Blynk.config(BLYNK_AUTH_TOKEN);
  Blynk.connect();
  Serial.println("Berhasil Terhubung ke Blynk Cloud!");
  Blynk.virtualWrite(V8, "00:00:00");

  // 3. Konfigurasi OTA
  ArduinoOTA.setHostname("ESP32-Jonathan-FinalUTS");
  ArduinoOTA.setPassword("admin123");

  ArduinoOTA.onStart([]() { Serial.println("\n MENERIMA UPDATE OTA..."); });
  ArduinoOTA.onEnd([]() { Serial.println("\n OTA SUKSES! RESTARTING...");
});
  ArduinoOTA.onProgress([](unsigned int progress, unsigned int total) {
    Serial.printf("Progress: %u%%\r", (progress / (total / 100)));
```

```
  });

  ArduinoOTA.begin();
  Serial.println("\n--- BLYNK + TIMER + OTA SIAP 100% ---");
}

//Loop Utama

void loop() {
  Blynk.run();         // Jaga koneksi HP <--> ESP32
  ArduinoOTA.handle(); // Jaga pintu nirkabel untuk update kodingan

  // Indikator Siaga di Serial
  if (!isTimerAktif && (millis() - waktuTeksTerakhir >= jedaTeks)) {
    waktuTeksTerakhir = millis();
    Serial.println("[INFO] Sistem Standby... OTA dan Blynk Aktif.");
  }

  // --- LOGIKA HITUNG MUNDUR FORMAT JAM:MENIT:DETIK ---
  if (isTimerAktif) {
    unsigned long waktuBerjalan = millis() - waktuMulaiTimer;

    if (waktuBerjalan < durasiTimer) {
      long sisaTotalDetik = (durasiTimer - waktuBerjalan) / 1000;

      if (sisaTotalDetik != sisaWaktuTerakhir) {
        int h = sisaTotalDetik / 3600;
        int m = (sisaTotalDetik % 3600) / 60;
        int s = sisaTotalDetik % 60;

        char teksWaktu[10];
        sprintf(teksWaktu, "%02d:%02d:%02d", h, m, s);

        Blynk.virtualWrite(V8, teksWaktu);
        sisaWaktuTerakhir = sisaTotalDetik;
      }
    } else {
      // Waktu Habis
      isTimerAktif = false;
      isMotorRunning = false;
      applyMotor();

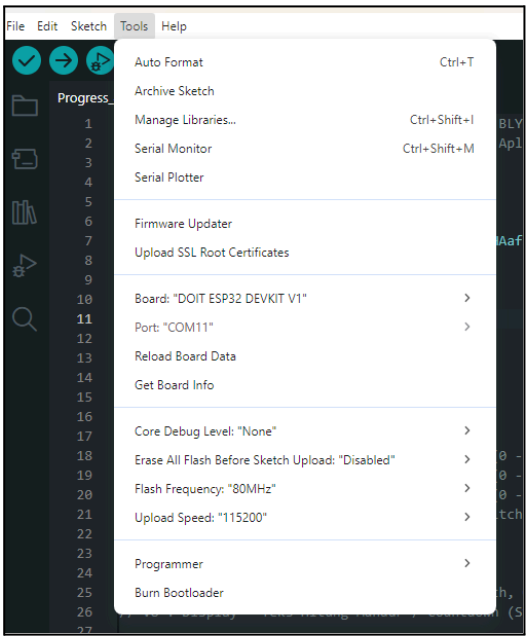
      Blynk.virtualWrite(V7, 0);
      Blynk.virtualWrite(V8, "SELESAI");
      Serial.println("\n>>> [TIMER] Waktu habis, Motor otomatis MATI.");
    }
  }
}
```

Pada arsitektur tahap otomasi tingkat lanjut ini, fondasi kendali aktuator dan algoritma *timer* asinkron millis() dipertahankan secara utuh, namun diekspansi dengan injeksi pustaka <ArduinoOTA.h>. Konfigurasi basis OTA diinisialisasi di dalam fungsi setup() melalui penetapan nama *hostname* jaringan dan pengamanan jalur transmisi menggunakan parameter setPassword(). Selama siklus loop() utama berjalan, fungsi ArduinoOTA.handle() dieksekusi secara konstan berdampingan dengan fungsi Blynk.run(). Mekanisme pendengar (*listener*) nirkabel ini memungkinkan mikrokontroler untuk senantiasa siaga memantau permintaan *flashing* sistem baru (OTA) tanpa membekukan iterasi waktu hitung mundur yang sedang aktif beroperasi di latar belakang.

4.6 Hasil Pengujian & Analisis

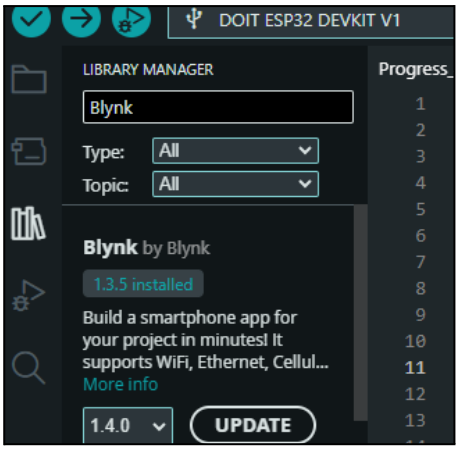
a. Proses Kompilasi dan Upload Program

Sebelum melakukan kompilasi dan pengunggahan (*upload*) program, pastikan terlebih dahulu bahwa *port* komunikasi awal telah terhubung dengan benar ke kit ESP32 menggunakan kabel USB. Pastikan jenis *board* yang dipilih pada menu *Tools* adalah DOIT ESP32 DEVKIT V1 dengan kecepatan unggah (*Upload Speed*) sebesar 115200.



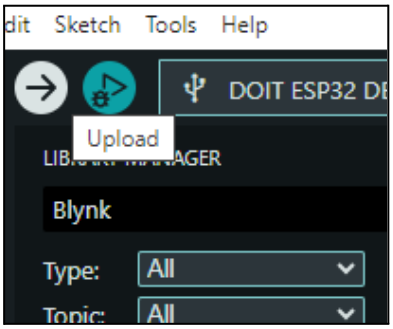
Gambar 2. Menu Tools Pemilihan Board dan Port pada Arduino IDE.

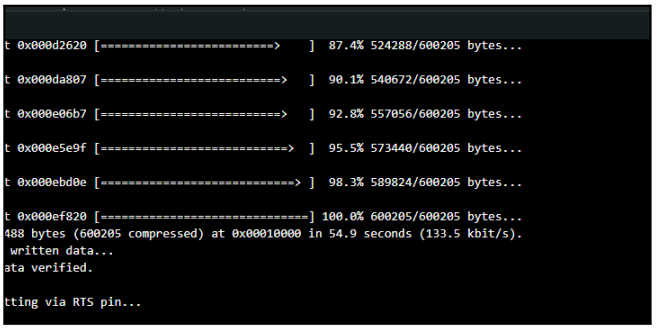
Selain itu, pastikan kembali bahwa *library* "Blynk by Blynk" telah terinstal pada *Library Manager*, sesuai dengan tahap persiapan (*setup*).



Gambar 3. Pengecekan Library Blynk pada Arduino IDE.

Hal krusial lainnya yang wajib dipastikan adalah jaringan Wi-Fi (atau *hotspot* seluler) yang digunakan untuk mengoneksikan ESP32 beroperasi pada pita frekuensi 2,4 GHz. Modul nirkabel pada ESP32 tidak mendukung konektivitas pada frekuensi 5 GHz. Jika pengaturan jaringan sudah dipastikan benar, langkah selanjutnya adalah menekan tombol *Upload* untuk menanamkan kode (*firmware*) ke dalam mikrokontroler.

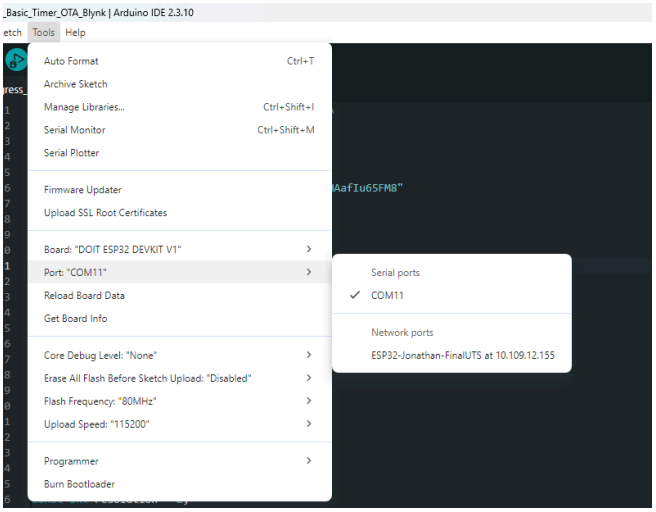




Gambar 4 dan 5. Eksekusi proses kompilasi dan pengunggahan (*upload*) program awal via kabel serial.

Tunggu hingga proses *upload* selesai. Jika kode berhasil diunggah tanpa galat (*error*), maka pada panel *Output* akan muncul log keterangan bahwa memori *flash* telah berhasil ditulis, ditandai dengan informasi "Wrote ... bytes" dan eksekusi *Hard resetting via RTS pin*.

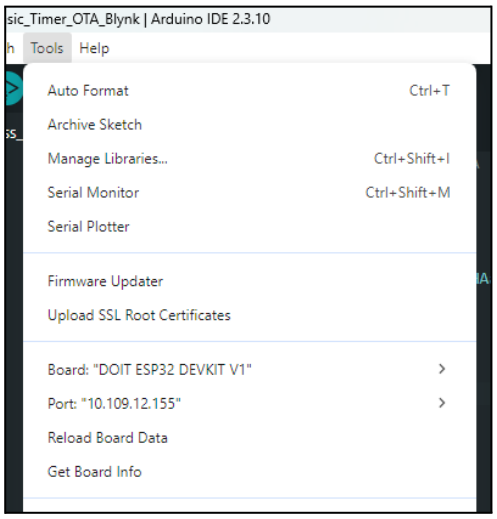
Selanjutnya, bukti empiris bahwa kode telah berhasil dikompilasi dan fungsi OTA beroperasi dengan baik adalah munculnya port baru pada menu *Tools*, yakni *Network Port* (IP Address), seperti yang ditunjukkan pada Gambar 6.



Gambar 6. Indikator keberhasilan inisialisasi
OTA berupa munculnya *Network Port* (IP Address) pada menu *Tools*.

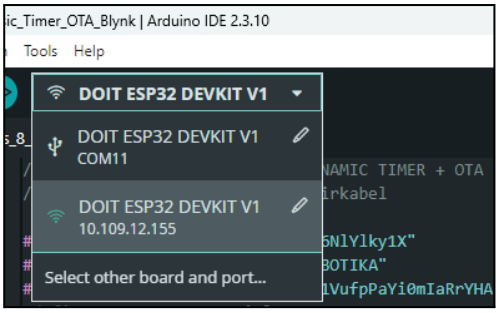
b. Pengujian

Berbeda dengan pengujian sebelumnya, eksperimen kali ini dilakukan dengan mengunggah (*upload*) ulang kode Progress 4 melalui *Network Port* untuk memverifikasi fungsionalitas pembaruan nirkabel. Langkah pertama adalah mengubah opsi *Port* yang sebelumnya menggunakan kabel serial (COM) beralih ke *Network Port* yang terdeteksi pasca-upload kode OTA pertama.



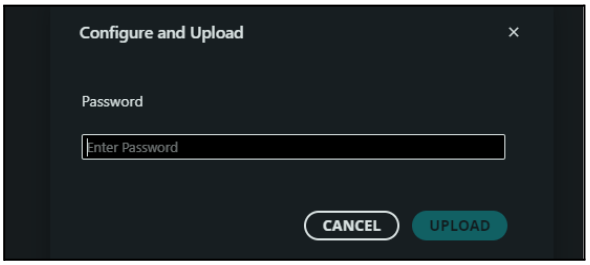
Gambar 7. Pemilihan *Network Port* untuk
memulai mode pembaruan *firmware* nirkabel (OTA).

Selain memastikan port jaringan telah terpilih, *Board* yang digunakan harus dipastikan tetap berada pada DOIT ESP32 DEVKIT V1.



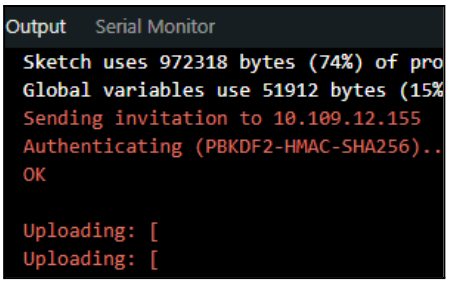
Gambar 8. Konfirmasi pemilihan identitas
Board yang disertai dengan simbol koneksi Wi-Fi.

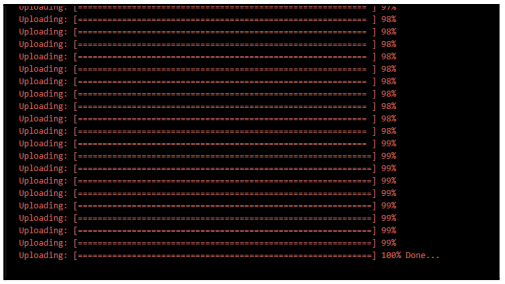
Setelah konfigurasi dipastikan benar, instruksi *upload* ulang dapat dieksekusi. Indikator utama bahwa parameter OTA berhasil diakses adalah munculnya kotak dialog otentikasi yang meminta pengguna untuk memasukkan kata sandi (*password*) sesaat setelah tombol *upload* ditekan. Kata sandi ini mengacu pada kredensial yang telah diinisialisasi secara mandiri di dalam kode program sebelumnya.



Gambar 9. Proses otentikasi dengan
memasukkan kata sandi (*password*) sistem OTA.

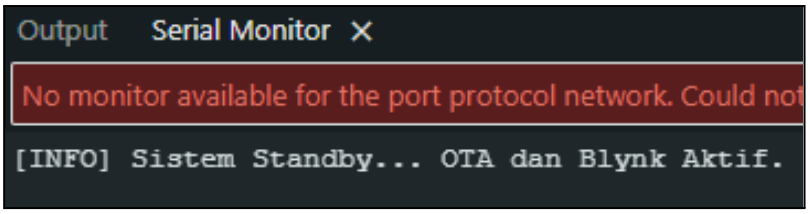
Pasca-otentikasi, proses kompilasi dan pengiriman data binari akan berjalan. Secara visual, perbedaan transmisi OTA dengan transmisi kabel fisik terlihat pada panel log *Output*; proses unggah via OTA akan menampilkan log persentase transmisi berwarna merah, sedangkan penggunaan kabel serial konvensional umumnya menampilkan teks instruksi berwarna putih.





Gambar 10 dan 11. Proses kompilasi dan pengunggahan
(*upload*) *firmware* nirkabel yang ditandai dengan indikator log berwarna merah.

Begitu pembaruan selesai 100% dan mikrokontroler melakukan *reboot*, antarmuka Serial Monitor tidak lagi dapat digunakan (karena tidak ada kabel fisik yang mentransmisikan data komunikasi COM port ke layar). Pada titik ini, seluruh kendali operasional, eksekusi perintah motorik, dan pemantauan visibilitas hitung mundur (*timer*) sepenuhnya didelegasikan secara absolut ke antarmuka aplikasi Blynk.



Gambar 12. Notifikasi indikator status siaga (*standby*) sistem OTA pada antarmuka Serial Monitor.

c. Analisis Keseluruhan

Berdasarkan serangkaian uji coba komprehensif, penyematan fitur *Over-The-Air* (OTA) pada sistem kendali motor berbasis antarmuka Blynk terbukti sukses beroperasi secara harmonis di bawah arsitektur asinkron. Mikrokontroler menunjukkan tingkat reliabilitas yang sangat tinggi dalam mengelola koneksi Wi-Fi secara ganda; ESP32 mampu menyiarkan IP Address untuk menerima injeksi *firmware* baru melalui jaringan tanpa memicu tabrakan data (*network collision*), sembari secara konsisten mengeksekusi algoritma otomasi *dynamic timer* berbasis millis().

Kemampuan untuk memanipulasi *firmware* mikrokontroler dan memantau status operasionalnya secara jarak jauh absolut ini membuktikan bahwa perangkat sistem tertanam (*embedded system*) yang dirancang telah memenuhi kualifikasi standar industri *Internet of Things* (IoT) murni. Hal ini secara efektif mengeliminasi dependensi fisik perangkat terhadap pemeliharaan maupun intervensi manual via konektor kabel konvensional.